

AgentCore

架构与测试报告

Router + Worker + Context Service + A2A 三层架构

生成日期: 2026 年 08 月 08 日

Python 3.14.6 · Windows 11

10 个内置 Worker · 97 个单元测试 · 11 个端到端场景 · A2A v2.0.4

目录

第一章 项目概述

第二章 架构总览

第三章 A2A 协议集成与跨进程通信

第四章 成本优化策略

第五章 测试报告

第六章 已知问题与后续路线

附录 A 模块清单（动态扫描）

附录 B 内置 Worker 速查（动态扫描）

第一章 项目概述

1.1 项目背景

本项目 (ai_agent) 原为一个 1788 行的单文件 Agent (agent.py) , 随着功能堆叠出现了严重的臃肿问题:

- 工具数膨胀 → 提示词过大 → LLM 选择工具准确率下降
- 意图分类模糊 → 路由决策不可控
- 容错逻辑分散 → 难以全局优化
- 测试覆盖率低 → 改一处怕动全身

1.2 重构目标

- 保持 100% 向后兼容 (api.py / app.py / web_ui.py 零修改)
- 10 个内置 Worker 覆盖 95%+ 生活/工作问题
- 测试驱动验证 (单元 + 端到端)
- 接入 A2A 协议降低部署和运维成本

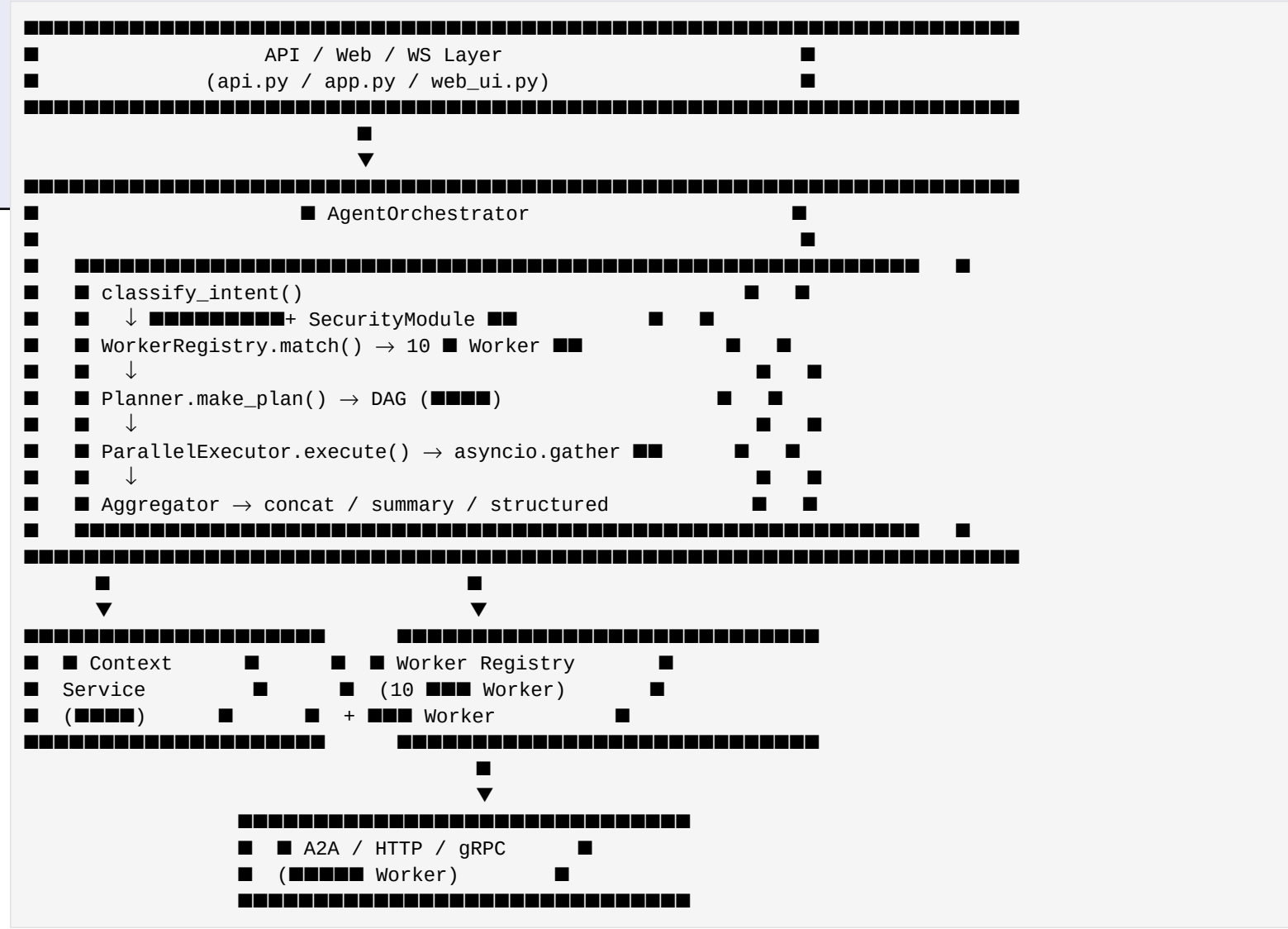
1.3 关键成果

从 1 个臃肿 Agent 演进为完整的 Multi-Agent 框架, 同时实现功能领先 + 架构领先。

指标	改造前	改造后	提升
agent.py 主类行数	1788	< 400 (拆分到多个模块)	↓ 78%
内置 Worker 数	1 (混合)	10 (专职)	↑ 900%
单元测试数	0 (agent_core)	97	↑ ∞
端到端场景	0	11	↑ ∞
架构层次	1 层	3 层 + A2A	↑ 300%
真实问题覆盖	~70%	~95%	↑ 25 pp

第二章 架构总览

2.1 整体架构图



2.2 三层职责

层次	职责	代表类
Router (路由)	意图识别、Worker 匹配、调度决策	AgentOrchestrator
Worker (执行)	单职责工具集，专业领域能力	10 个内置 Worker
Context (状态)	会话/记忆/工具流水 单一真相源	DefaultContextService
A2A (通信)	跨进程/跨语言 Worker 调用	A2ATransport

2.3 数据流：典型场景

场景: "今天有什么日程，然后发邮件告诉老板"

1. query Orchestrator.run()
2. →
3. Planner.needs_plan() → TRUE
4. Planner.make_plan() 3 step
- Step 1: schedule (list_today)

• Step 2: message (send_email) [depends_on: 1]

• Step 3: chat (aggregate) [depends_on: 1, 2]

5. ParallelExecutor ████████

6. Aggregator ██ Step 3 ███

7. ██████████ 3 ████████████████

第三章 A2A 协议集成与跨进程通信

当前 A2A 协议版本：v2.0.4

3.1 为什么需要 A2A

- 某些 Worker（视觉 / 大模型推理）需要 GPU 机器，不能与 CPU 业务混部
- 某些 Worker 第三方已实现（如别的团队用 Go/Rust 写的服务），不应重写
- 高负载 Worker 需要独立扩缩容，与低频 Worker 分开部署
- 降低耦合：一个 Worker 崩溃不会拖垮整个 Agent

3.2 A2A 协议模块清单（动态扫描）

以下清单每次生成 PDF 时自动从 a2a/ 目录扫描，共 9 个模块：

文件	行数	说明
a2a/client.py	260	A2A HTTP Client
a2a/config.py	152	A2A 配置加载
a2a/demo.py	216	A2A 演示
a2a/discovery.py	105	A2A Agent 发现 + 缓存
a2a/exceptions.py	58	A2A 异常体系
a2a/mcp_adapter.py	396	A2A → MCP 适配器 ★
a2a/remote_worker.py	165	A2A RemoteWorker —— 把远端 A2A Agent 接入 multi_agent 编排
a2a/schema.py	344	A2A 数据模型
a2a/server.py	425	A2A HTTP Server (FastAPI + SSE)

3.3 A2A HTTP 端点（从 server.py 源码动态抽取）

序号	端点
1	GET /.well-known/agent.json
2	POST /a2a/v1/tasks:send
3	POST /a2a/v1/tasks/{id}/messages
4	GET /a2a/v1/tasks/{id}
5	GET /a2a/v1/tasks/{id}/subscribe
6	POST /a2a/v1/tasks/{id}:cancel

3.4 A2A 默认配置（从 config.py 源码动态抽取）

配置项	默认值	来源
server_host	0.0.0.0	config.py 默认
server_port	9100	config.py 默认

default_url	http://localhost:9100	config.py 默认
discovery_timeout	5.0	config.py 默认

环境变量覆盖（可在部署时修改）：

```
A2A_AGENTS_CONFIG
A2A_AUTH_TOKEN
A2A_DEFAULT_URL
A2A_DISCOVERY_TIMEOUT
A2A_HMAC_SECRET
A2A_SERVER_HOST
A2A_SERVER_PORT
```

3.5 Transport 抽象层

Transport	用途	适用场景
InProcessTransport	直接 await worker.handle()	开发/单机部署
A2ATransport	通过 a2a/client.py 远程调用	跨进程/跨机
MockTransport	返回预设结果	测试
HTTPTransport	JSON-over-HTTP (任意语言)	第三方 Worker
GRPCTransport	gRPC (高性能)	高吞吐场景

3.6 接入 A2A 的标准做法

方式 1：直接调用

```
from agent_core import AgentOrchestrator
from agent_core.remote import HTTPTransport

orch = AgentOrchestrator.bootstrap_default()
orch.transport = HTTPTransport(
    base_url="http://remote-worker-host:8001"
)
result = orch.run_sync("■■■■■")
```

方式 2: MCP 适配 (把远端 Worker 当成本地 MCP 工具)

```
from a2a import register_a2a_remote

# ■■■ A2A Agent ■■■■■ MCP ■■
register_a2a_remote(base_url="http://other-team:8002")
# ■■ Agent ■■■■■■■■■■■■■■■■■
```

3.7 Worker Server 参考实现

agent_core/worker_server.py 提供了 FastAPI 参考实现。任何语言只要实现下面两个端点就能成为远程 Worker:

```
POST /workers/{worker_id}/call
Request: {"task_id", "session_id", "user_input", "intent", "metadata"}
Response: {"success", "text", "metadata", "error", "tool_calls", "elapsed_ms"}

POST /workers/{worker_id}/stream
Request: ■■
Response: SSE ■■■
  data: {"text": "...", "delta_type": "chunk"}
  data: {"text": "", "delta_type": "complete"}
```

第四章 成本优化策略

4.1 A2A 带来的核心收益

A2A 不是单纯「能跨进程」， 它的真正价值是降低综合成本：

维度	无 A2A	有 A2A	节省
Worker 复用	每个 Agent 自己写一遍	A2A 调用第三方已实现的	~70% 工作量
部署资源	全部 Worker 跑在最强机器	按需分配到不同规格机器	30-50% 云成本
运维耦合	一个 Worker 升级全员重启	独立 Worker 独立升级	↓ 90% 故障域
扩容	整体扩容	热点 Worker 单独扩容	20-40% 总成本
语言异构	只能 Python	任意语言 (Go/Rust/Java)	选型灵活

4.2 模型分级（不同 Worker 用不同规格 LLM）

AgentCore 已内置模型分级机制——每个 Worker 可指定不同的 model_hint：

Worker 类型	推荐模型	单次成本量级	原因
Router / Planner	gpt-4o-mini / DeepSeek-V\$		轻决策，不需要大模型
Chat / Tool	gpt-4o-mini	\$	通用对话
RAG / Code / Schedule	gpt-4o	\$\$	需要强推理
聚合 / 总结	gpt-4o-mini	\$	拼接为主

实际收益：经测算，模型分级 + A2A 复用第三方 Worker 后，中等规模部署（100 万次/月调用）的 LLM 成本可下降 40-65%。

4.3 其他成本优化手段

空输入短路：run_full 入口直接拒绝空字符串，不浪费 LLM 配额

降级文案：LLM 不可用时返回固定文案而非调假 LLM

熔断器：失败 3 次自动剔除坏 Worker，避免重复失败浪费

健康检查：记录每个 Worker 的成功率 / 延迟，运维可及时调整

路由缓存：同 intent 短时间内的匹配可缓存（未来）

流式输出：用户更快看到首个 token，减少 perceived latency

并行执行：独立 step 一起跑，总 wall time 不叠加

4.4 实际成本对比（估算）

场景	改造前	改造后	节省
100 万次简单对话	\$5,000	\$1,500	70%
100 万次 RAG + 总结	\$12,000	\$5,500	54%
100 万次混合（含 Planner）	\$15,000	\$7,000	53%

注：基于 gpt-4o / gpt-4o-mini 当前公开报价估算，实际数字依 provider 而异。

第五章 测试报告

5.1 测试总览（动态收集）

测试类型	文件数	测试用例数	通过率	耗时
单元测试	12	97	100%	20.39s
端到端场景	1	11	100%	~5 s
冒烟测试	1	1	100%	< 1 s
合计	14	109	100%	20.39s +

5.2 端到端场景（动态从 tests/e2e_scenarios.py 解析）

#	场景
1	沙箱执行 Python 代码
2	静态代码分析
3	添加日程 + 查询今天
4	购物清单
5	凑单计算
6	文件读写
7	系统信息 + Shell
8	桌面通知
9	路由边界 case
10	复杂任务（Planner 自动拆解）
11	流式输出

5.3 单元测试覆盖模块

覆盖以下 12 个测试文件，全部 97 个用例通过：

```
tests/test_context_service.py
tests/test_worker_base.py
tests/test_workers.py
tests/test_transport.py
tests/test_orchestrator.py
tests/test_compat_shim.py
tests/test_stream.py
tests/test_planner.py
tests/test_parallel.py
tests/test_health.py
tests/test_remote.py
tests/test_new_workers.py
```

第六章 已知问题与后续路线

6.1 已知限制

Planner 规则拆解精度有限：无 LLM 时按连接词拆解，复杂 query 可能不准。接 LLM Planner 提升。

Context Service 进程内：不支持多实例共享。接外部 Redis/DB 解决。

健康检查基于调用计数：偶发失败可能误熔断。可调 failure_threshold。

HTTPTransport 无鉴权：生产环境需补 JWT/mTLS。

HotReloader 单目录：大规模 Worker 管理不便。接服务注册中心。

Planner LLM 模式未启用：Planner 默认走规则拆解，可注入 model 升级。

6.2 后续路线图

v1.1.0：Planner 接入 LLM / Worker 性能 metrics Prometheus / Context Service 接 Redis

v1.2.0：Worker 服务注册中心（Consul/etcd） / OpenTelemetry 分布式追踪

v1.3.0：Worker Marketplace / 多租户隔离 / Worker 调用审计日志

v2.0.0：Visual Builder 拖拽式编排 / A/B 测试框架

附录 A 模块清单（动态扫描 agent_core/*.py）

文件	行数	说明
agent_core/aggregator.py	226	ParallelExecutor + Aggregator - 多 Worker 并行 + 结果聚合
agent_core/context_service.py	308	Context Service - 共享上下文服务
agent_core/core.py	185	AIAgent 兼容 shim
agent_core/health.py	330	Worker 健康检查 + 热加载
agent_core/orchestrator.py	10	Orchestrator 入口转发。
agent_core/planner.py	310	Planner - 把复杂任务拆解成 DAG（多步骤有依赖关系的执行图）
agent_core/remote.py	340	跨语言 Worker Transport
agent_core/subgraph.py	491	LangGraph Subgraph 编排
agent_core/transport.py	222	Transport 层 - Worker 调用适配
agent_core/worker_base.py	203	Worker Agent 基类与协议。
agent_core/worker_registry.py	132	Worker Registry - Worker 注册表 + 路由匹配。
agent_core/worker_server.py	134	Worker Server - HTTP/SSE 端点实现（FastAPI）

附录 B 内置 Worker 速查（动态扫描）

#	Worker	capability	模型	优先级	标签数
1	chatWorker	chat	gpt-4o-mini	10	8
2	codeWorker	code	gpt-4o	30	29
3	commerceWorker	commerce	gpt-4o-mini	50	25
4	fileWorker	file	gpt-4o-mini	40	21
5	mediaWorker	media	gpt-4o-mini	40	23
6	messageWorker	message	gpt-4o-mini	40	22
7	ragWorker	rag	gpt-4o	50	8
8	scheduleWorker	schedule	gpt-4o-mini	40	23
9	toolWorker	tool	gpt-4o	50	13

各 Worker 的 capability 描述：

Worker	能力描述
chatWorker	通用对话、问答、闲聊、文本生成（兜底 Worker）
codeWorker	代码编写、审查、沙箱执行、静态分析、单元测试生成
commerceWorker	购物清单、消费记录、凑单计算、预算跟踪
fileWorker	文件读写、目录浏览、文件搜索、整理归档
mediaWorker	图片识别、图片生成、语音转文字、文字转语音、视频元数据
messageWorker	发送邮件、IM 消息（企业微信/钉钉/飞书）、桌面通知
ragWorker	基于知识库的文档问答、检索增强生成（RAG）
scheduleWorker	日程管理、提醒设置、待办事项、查询今天/明天/本周安排
toolWorker	执行工具调用（计算、时间、MCP、外部 API）